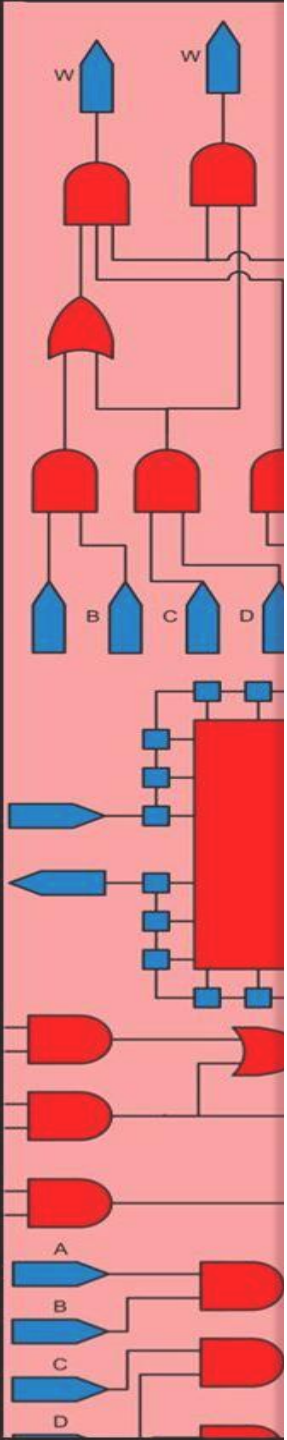


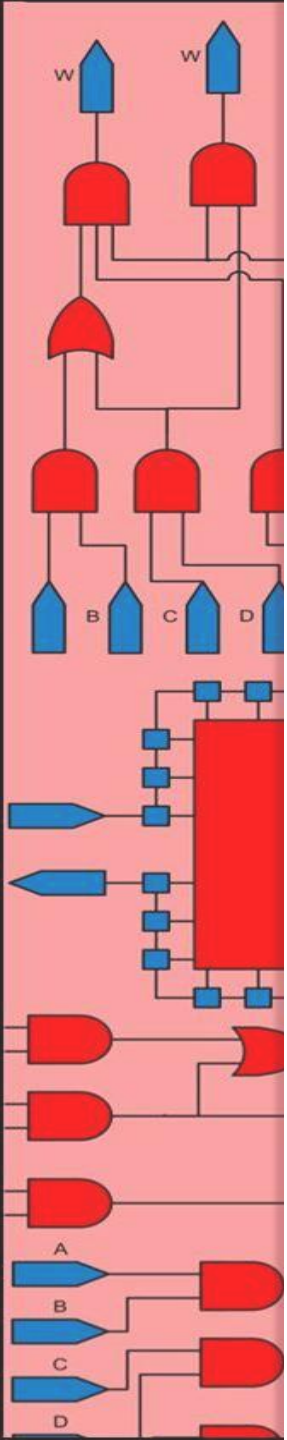
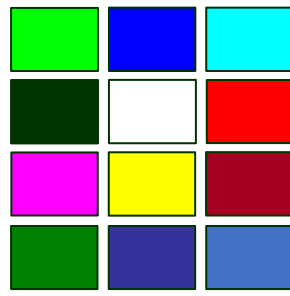
Topic 2

Combinational Components SystemVerilog Constructs



Combinational Components

- Decoder
- Multiplexer
- Comparator
- Adder
- Shifter
- ALU
- Case Studies
- **SystemVerilog Constructs**



SystemVerilog Constructs

- Operators

Bitwise operators

- NOT : $\sim a$
- AND : $a \& b$
- OR : $a | b$
- XOR : $a \wedge b$
- XNOR : $a \sim \wedge b$

Logical operators

- NOT: $!a$
- AND: $a \&\& b$
- OR: $a || b$

Reduction operators

- AND: $\&a$
- NAND : $\sim\&a$
- OR: $|a$
- NOR : $\sim|a$
- XOR: $\wedge a$

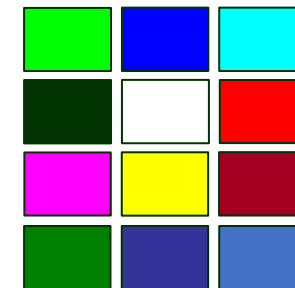
Shift Operators

- $\gg$
- $\ll$
- $\ggg$
- $\lll$

Comparison operators

- Relational
 - $a > b$
 - $a < b$
 - $a \geq b$
 - $a \leq b$
- Equality
 - $a == b$
 - $a != b$

SystemVerilog Constructs



- *logic* data type
 - single-bit or vector ports & signals
 - *unsigned* & *signed*

4- Value Logic Model

```
module mux8to1 (  
    input  logic [7:0] J,  
    input  logic [2:0] S,  
    output logic      W  
);  
  
    assign W = J[S];  
  
endmodule
```

Value Level	HW Condition
0	Logic zero, False
1	Logic one, True
X	Bad situations, Unknown
Z	High impedance

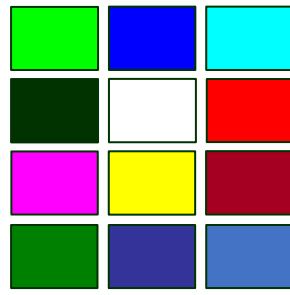
```
input  logic signed [7:0] a_signed,  
input  logic signed [7:0] b_signed,  
input  logic        [7:0] a_unsigned,  
input  logic        [7:0] b_unsigned,
```

SystemVerilog Constructs

- Primitive Instances
 - Transistor-level
 - Gate-level

```
nmos #delay [instance_name] (d, s, g)
```

```
or #10 R1(v, j, k);  
not #5 N1(ai, a), N2(bi, b), N3(si, s);  
and #15 A1(j, ai, bi, s);  
and #15 A2(k, a, b, si);
```



SystemVerilog Constructs

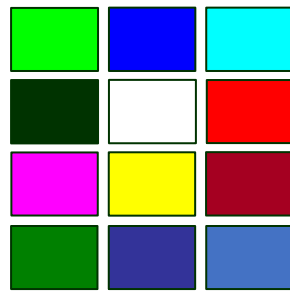
- *assign* statement,
 - Boolean, ?:

Concurrent Process

Executed whenever the **right-hand side** changes

```
assign S = a ^ b ^ cin;  
assign Co = (a & b) | (a & cin) | (b & cin);
```

```
assign w = (s == 2'b00) ? a :  
           (s == 2'b01) ? b :  
           (s == 2'b10) ? c :  
           (s == 2'b11) ? d : 1'bx;
```



SystemVerilog Constructs

- *always_comb*

Can contain complex procedural code, e.g., *if-else*, *case*, *for* loop

Executed whenever any signal on the **right-hand side** or any **condition** changes

```
always_comb begin
```

```
  w = 8'b0;
```

```
  c = 1'b0;
```

```
  case (f)
```

```
    3'd0: {c, w} = A + B;
```

```
    3'd1: {c, w} = {A, 1'b0};
```

```
    3'd2: w      = (A > B) ? A : B;
```

```
    3'd3: w      = A & B;
```

```
    3'd4: w      = A | B;
```

```
    3'd5: w      = ~A + 1'b1;
```

```
    3'd6: w      = A;
```

```
    3'd7: w      = ~A;
```

```
    default: w   = 8'd0;
```

```
  endcase
```

```
end
```

Input Rule → Implicit Sensitivity list

Output Rule

A concurrent process on the outside and a procedural block on the inside.

SystemVerilog Constructs

- *module* instances
 - Named port association
 - Positional port association

Named port association

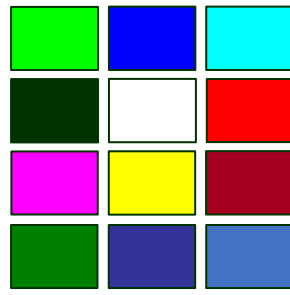
```
half_adder ha1(.a(a),.b(b),.s(s1),.c_out(c1));  
half_adder ha2(s1,c_in,c2,s);
```

Positional port association

SystemVerilog Constructs

- *generate-for* loop for iterative structures

```
genvar i;  
generate  
    for (i = 0; i < SIZE; i = i + 1) begin : full_adders  
        FA_v1 fa (a[i], b[i], c[i], s[i], c[i+1]);  
    end  
endgenerate
```



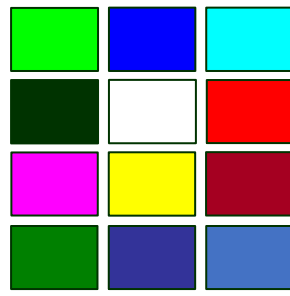
Combinational components

- Parameterized module

```
module mux_Nto1 #(
    parameter N = 4
) (
    input  logic [N-1:0] J,
    input  logic [$clog2(N)-1:0] s,
    output logic W
);

    always_comb begin
        W = J[s];
    end
endmodule
```

Parameter definition



Combinational components

- Testbench

- *initial* statement

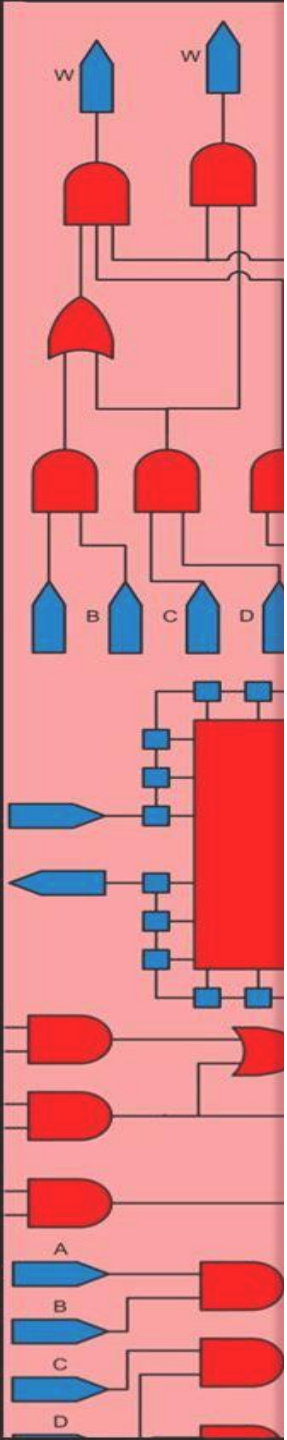
A concurrent process

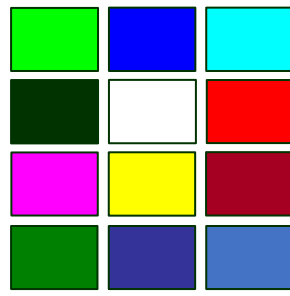
- Built-in functions

Executed only once at simulation time 0

- Input Data generation
 - Deterministic data
 - Random data: \$urandom, \$urandom_range(min, max), ...
- Output Analysis
 - \$display
 - Waveform
- To pause or terminate simulation: \$stop, \$finish
- To return the current simulation time: \$ time
- To wait for a specific time: #delay

Non-synthesizable SystemVerilog Constructs





Conclusions

- Operators
- *logic* data type
- Primitive Instances
- *assign* statement
- *always_comb* Process
- *module* instances & binding
- Parameterized module
- *generate-for* loop for iterative structures
- Testbench constructs

